Institut Notre-Dame Beauraing

Année scolaire : 2025-2026
UAA13 : Conception et gestion de projet
Titulaire du cours : Lucas Embrechts
6e Technique de Transition – Informatique

UAA13

Conception et gestion de projet

Nom / Prénom : .....

Classe : .....

		Pondération	Pondération finale
Q1	Implémentation du projet	/100	/45
Q2	Dossier de conception	/100	/25
Q3	Défense Orale	/100	/30
TOTAL /100			

1. Description générale

Les élèves ont pour objectif de réaliser :

- une plateforme WEB codée en PHP, HTML, CSS et JavaScript ;
- une base de données MySQL hébergée en local ;
- un dossier de conception, documentant le travail fourni tout au long de l'année.

Ce projet sera clôturé par une présentation de groupe ainsi que par une défense orale individuelle.

Le projet choisi sera approché comme un réel projet informatique dans le sens où il suivra la méthodologie agile « SCRUM »[1].

À savoir :

- l'identification de « user stories » ;
- la rédaction d'un « product backlog » ;
- le découpage du projet en différents « sprints » ;
- la planification de ces sprints sur l'année ;
- la réalisation de « sprint backlog » ;
- l'organisation de « daily scrum » ;
- l'organisation de « sprint review » ;
- l'organisation de « sprint retrospective ».

Des informations complémentaires sur SCRUM sont disponibles sur la plateforme « info305 ».

1 Cette méthodologie sera adaptée pour correspondre aux contraintes organisationnelles liées au déroulement de l'année scolaire.

2. Planification du projet sur l’année et délivrables

Le projet débute la première semaine de cours après les congés d’hiver et se déroulera jusqu’à la fin de l’année scolaire.

Trois sprints seront organisés durant cette période à intervalles réguliers.

Dans le cadre scolaire, les daily scrums seront adaptés et organisés sous la forme d'une réunion hebdomadaire. Ces réunions auront lieu chaque lundi pendant 15 minutes (sauf exceptions).

À la fin de chaque sprint auront lieu les sprint reviews, sprint retrospectives ainsi que les sprint backlogs. De plus, à la fin du premier et du deuxième sprint, une remise intermédiaire du dossier de conception sera demandée.

La planification complète est disponible dans un document dédié sur Classroom.

3. Dossier de conception et d’analyse du projet

Le dossier de conception sert à documenter et structurer les aspects techniques, fonctionnels et organisationnels d'un projet, afin de guider son développement et d'assurer sa compréhension et sa reproductibilité. **Il s’agit d’un document essentiel, car il constituera une base de travail pour les élèves des années suivantes qui reprendront et poursuivront le projet.**

Ce dossier reprendra :

1. La définition du projet
2. Analyse et conception
3. Implémentation
4. Guide d’utilisation
5. Rétrospective
6. Perspectives

3.1. Définition du projet

Cette section reprend la définition complète du projet. En d’autres termes, la description des besoins, les délivrables ainsi que les technologies utilisées.

3.2. Analyse et conception

3.2.1. Tâches techniques

La liste des différentes tâches techniques identifiées.

3.2.2. User stories

La liste des différentes user stories identifiées.

3.2.3. Product backlog

Le product backlog du projet.

3.2.4. Interfaces

Pour chaque user story, une interface (ou des exemples de dialogue) du programme.

3.2.5. Description des données

Le schéma entités-associations ainsi que le schéma relationnel de la base de données.

Pour ce schéma, pour chaque table :

- une description générale des données qu'elle contient
- la structure des données qu'elle contient c'est-à-dire le type de chaque donnée (ex. un entier) et la signification de chaque donnée (ex. l'age d'une personne) et leurs valeurs possibles le cas échéant

3.3. Implémentation

Cette section présente le code source de manière documentée et approfondie. Il ne s'agit pas simplement d'un copier/coller du code, mais d'une analyse détaillée de certains aspects spécifiques : des sections complexes nécessitant une explication approfondie ou encore des éléments dont vous êtes particulièrement fiers et qui méritent une attention particulière.

Pour une meilleure compréhension, il est conseillé d'accompagner les sections de code d'explications illustrées par des captures d'écran. Ces captures devraient montrer le résultat visuel ou fonctionnel correspondant dans l'application. Si certaines fonctionnalités reposent sur des algorithmes complexes ou une logique spécifique, des schémas explicatifs peuvent également être ajoutés pour en faciliter la compréhension.

3.4. Guide d'utilisation

Cette section présente un guide d'utilisation qui permette à un utilisateur lambda de faire fonctionner l'application sur un an. Ce guide doit être illustré par des captures d'écran de votre plateforme. Les utilisateurs doivent être autonomes.

3.5. Rétrospective

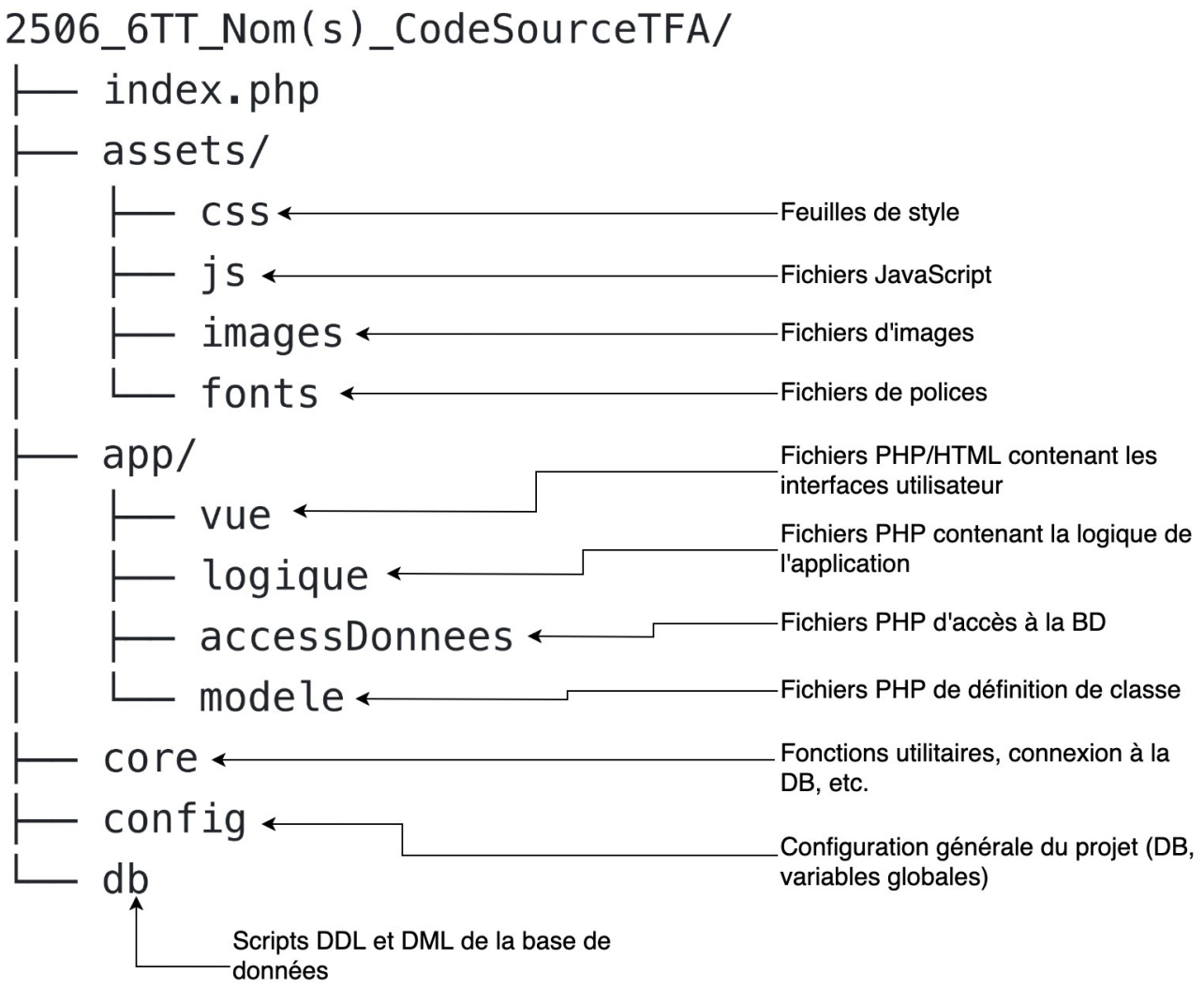
Une rétrospective du projet : ses forces, ses faiblesses, etc. Si certaines fonctionnalités n'ont pas pu être implémentées, c'est également ici que les raisons doivent être expliquées.

3.6. Perspectives

Les améliorations du programme : que peut-on améliorer ? que peut-on imaginer en plus ? etc.

4. Code source

- La plateforme doit être développée avec la technologie PHP / Javascript / CSS / HTML.
- Toutes les fonctionnalités développées en PHP doivent respecter le paradigme de programmation orientée objet.
- La plateforme doit être dynamique c'est-à-dire alimentée par une base de données MySQL.
- La plateforme doit reposer principalement sur des traitements côté serveur, ce qui implique une prédominance du code en PHP par rapport à JavaScript.
- La plateforme doit suivre le principe de découpe en couche et plus particulièrement l'architecture trois tiers.
- L'entièreté du code doit être développé par les membres de l'équipe. Chaque fichier doit présenter un commentaire en en-tête avec le nom de la personne ayant codé ce fichier (si cette partie a été codée en duo, les noms des deux, auteurs doivent être indiqués).
- En tout temps, l'interface présentera un pied de page (*footer*) fixe reprenant les noms des membres de l'équipe, le nom du projet et l'année académique. Le pied de page doit apparaître sur toutes les pages de la plateforme, au bas de l'écran, sans gêner l'affichage du contenu principal.
- Les parties plus complexes du code doivent être commentées de manière à permettre à un utilisateur lambda de comprendre et de reprendre facilement le code. C'est-à-dire que pour chaque partie de code répondant à une tâche spécifique, le développeur doit décrire en commentaire ce que le code réalise. Tout commentaire inutile sera sanctionné dans la cotation. Le commentaire doit être précis, pertinent et apporter un éclairage utile sur la logique de la tâche.
- La plateforme et la base de données doivent pouvoir fonctionner en local sur l'outil XAMPP.
- L'ensemble du code source doit être hébergé dans un dépôt GIT sur la plateforme GitHub ; en tout temps, ce dépôt doit être accessible au professeur.
- Une architecture bien spécifique, présentée à la page suivante, doit être respectée.


2506_6TT_Nom(s)_CodeSourceTFA/
index.php
assets/
css
Feuilles de style
js
Fichiers JavaScript
images
Fichiers d'images
fonts
Fichiers de polices
app/
vue
Fichiers PHP/HTML contenant les interfaces utilisateur
logique
Fichiers PHP contenant la logique de l'application
accessDonnees
Fichiers PHP d'accès à la BD
modele
Fichiers PHP de définition de classe
core
Fonctions utilitaires, connexion à la DB, etc.
config
Configuration générale du projet (DB, variables globales)
db
Scripts DDL et DML de la base de données

5. Système d'evaluation

5.1. Evaluation du travail journalier

Le formatif (travail journalier) concerne :

- les « daily scrum »
- les « sprint reviews »
- les « sprint retrospectives »
- les « sprint backlogs »
- les différentes remises intermédiaires du dossier de conception (c'est-à-dire non finalisé)
- la défense orale en groupe du projet

5.2. Evaluation sommative

La remise finale du projet constitue l'évaluation de l'UAA13.

Le projet est évalué sur base :

- d'une défense orale (30%)
- du dossier de conception (25%)
- du code source (45%)

Les différentes grilles d'évaluation sont disponibles à la fin de ce document.

Évaluation du dossier de conception

Le dossier de conception doit être complet. La mise en page, l'orthographe et la qualité de la rédaction seront prises en compte dans l'évaluation.

Évaluation du programme

Le programme sera évalué selon les points suivants :

- le programme s'exécute correctement au niveau des fonctionnalités attendues,
- le code source est de qualité.

Des bonus seront attribués pour les fonctionnalités supplémentaires non mentionnées dans le dossier.

Toutefois, ces bonus ne seront attribués que si la note du travail de base atteint les 80%.

Évaluation orale

L'évaluation orale consiste à présenter le projet oralement devant les superviseurs. La présentation se déroulera comme suit :

1. Présentation du logiciel

2. Présentation du code source
3. Questions / Réponses sur le logiciel, le code source et le cahier des charges

La présentation du logiciel ainsi que la présentation du code source ne devra pas dépasser 10 minutes.

Dans le cas où le projet est réalisé par deux élèves, ces derniers doivent impérativement démontrer leur participation au projet. À cette fin, le code source devra être documenté de telle manière que l'on puisse identifier clairement l'auteur de chaque ligne de code.

Lors de l'épreuve orale, les questions sur le code source ne porteront que sur le code écrit par l'élève. Par contre, les questions sur le cahier des charges porteront sur l'entièreté de ce dernier.

Attention : le niveau d'exigence sera supérieur pour les projets réalisés à deux, tant au niveau de la rédaction du cahier des charges qu'au niveau de la réalisation du programme.

Les différentes grilles d'évaluation sont disponibles à la fin de ce document.

6. Remise finale et nomenclature

Lors de la remise finale vous remettrez une archive compressée nommée **`2606_6TT_Nom(s)_TFA.zip`** comprenant :

1. Le dossier de conception et d'analyse du projet en .pdf

   Nomenclatures : **`2606_6TT_Nom(s)_DossierConceptionTFA.pdf`**
2. Un dossier nommé **`2606_6TT_Nom(s)_CodeSourceTFA`** contenant le code source ainsi que tous les fichiers nécessaires au bon fonctionnement du programme.

Aucun retard ne sera accepté sauf cas de force majeure. En cas de non remise à la date indiquée, une cote nulle sera attribuée.

7. Modalités de récupération

7.1.1. Conditions d'accès

La récupération est autorisée si :

- L'élève a participé au projet et a remis une première version exploitable (même incomplète).
- Les manquements relèvent d'améliorations possibles (fonctionnalités, dossier, préparation orale) et non d'une absence de travail.

7.1.2. Éléments à représenter

Selon les lacunes constatées, la récupération peut porter sur un ou plusieurs éléments :

- Le code / fonctionnalités attendues.
- Le dossier de conception (mise à jour et corrections).

- La défense orale individuelle.

7.1.3. Exigences minimales

- La version représentée doit corriger explicitement les points signalés lors de la première évaluation.
- Toute amélioration doit rester cohérente avec le cahier des charges et la méthodologie de travail demandée.
- Les livrables sont remis avec la même rigueur de présentation (structure, clarté, orthographe, etc.).

8. Indicateurs de qualité

8.1. La capacité fonctionnelle

« Le programme fait-il ce qu'il est censé faire ? »

La capacité fonctionnelle est la capacité qu'ont les fonctionnalités d'un logiciel à répondre aux exigences et besoins explicites ou implicites des usagers. En font partie la précision, l'interopérabilité, la conformité aux normes et la sécurité.

Exemple : Dans un jeu de Snake , si le serpent mange une pomme, son corps doit s'allonger d'un bloc et le score doit augmenter de 1 point. Si le score ne bouge pas, la capacité fonctionnelle est défaillante.

8.2. La facilité d'utilisation

« Comment utiliser le programme sans devoir lire un manuel de 50 pages ? »

Elle porte sur l'effort nécessaire pour apprendre à manipuler le logiciel. En font partie la facilité de compréhension, d'apprentissage et d'exploitation et la robustesse. Un autre exemple, une utilisation incorrecte n'entraîne pas de dysfonctionnement.

Exemple : Au lieu d'écrire « Appuyez sur la touche 34 pour sauter », utilisez les flèches du clavier ou les touches ZSQD qui sont naturelles pour un joueur. L'ajout d'un bouton « Rejouer » bien visible après une partie perdue améliore aussi cet indicateur.

8.3. La fiabilité

« Le programme plante-t-il ou s'arrête-t-il brusquement ? »

C'est-à-dire la capacité d'un logiciel de rendre des résultats corrects quelles que soient les conditions d'exploitation. En font partie la tolérance aux pannes - la capacité d'un logiciel de fonctionner même en étant handicapé par la panne d'un composant (logiciel ou matériel).

Exemple : Si un élève entre son nom dans le menu et qu'il tape des symboles bizarres (comme @#$%), le jeu ne doit pas s'éteindre ou afficher une page blanche. Il doit rester stable quelles que soient les actions imprévues de l'utilisateur.

8.4. La performance

« Le programme est-t-il une usine à gaz ? »

C'est-à-dire le rapport entre la quantité de ressources utilisées (moyens matériels, temps, personnel), et la quantité de résultats délivrés. En font partie le temps de réponse, le débit et l'extensibilité - capacité à maintenir la performance même en cas d'utilisation intensive.

Exemple : Si un jeu met 10 secondes à charger une image ou si le personnage se déplace avec des saccades (le « lag »), la performance est mauvaise. Un bon code JavaScript doit permettre au jeu de tourner de façon fluide, même sur un vieil ordinateur de l'école.

8.5. La maintenabilité

« Sera-t-il facile de modifier ou d'améliorer le code plus tard ? »

Elle mesure l'effort nécessaire à corriger ou transformer le logiciel. En font partie l'extensibilité, c'est-à-dire le peu d'effort nécessaire pour y ajouter de nouvelles fonctions.

Exemple : Si vous avez utilisé une variable `vitesse = 5` au début de votre code, vous pouvez changer la difficulté du jeu en modifiant un seul chiffre. Si vous avez écrit « 5 » manuellement à 100 endroits différents dans votre code, votre projet n'est pas maintenable.

8.6. La portabilité

« Le programme peut-il fonctionner partout ? »

C'est-à-dire l'aptitude d'un logiciel de fonctionner dans un environnement matériel ou logiciel différent de son environnement initial. En font partie la facilité d'installation et de configuration dans le nouvel environnement.

Exemple **:** un jeu doit fonctionner de la même manière si vous y jouer sur une console de Nintendo, Sony ou Microsoft.

GRILLE D'EVALUATION

IMPLEMENTATION DU PROJET

Grille d'évaluation					
Critères	Indicateurs	Points	Acquisition		
			A	N-A	E-A
Respect des normes, règles et consignes	A respecté les délais (MALUS) A remis le projet selon une nomenclature correcte (MALUS)				
Maitrise technique et processus	Maitrise des techniques de programmation Le projet ne présente aucun bug L'élève a amené des éléments nouveaux (recherches) Le code est documenté Le code est optimisé et permet sa maintenabilité Le code est de qualité				
Produit fini	Le projet représente un ensemble fonctionnel. L'élève a implémenté tous les contenus énoncés dans son dossier de conception. Le projet est original, les règles graphiques sont respectées et cohérentes. Le projet est ergonomique et facile à prendre en main.				
Communication	Le projet ne présente aucune faute d'orthographe.				

Pondération	/100

GRILLE D'EVALUATION

DOSSIER DE CONCEPTION

Grille d'évaluation				
Critères	Indicateurs	Acquisition		
		A	N-A	E-A
Respect des normes, règles et consignes	A respecté les délais (MALUS) A remis le fichier selon une nomenclature correcte (MALUS)			
Produit fini	Le dossier est complet. • Définition du projet • Analyse et conception • Implémentation • Guide d'utilisation • Rétrospective • Perspectives Le dossier explique les choix de conception de manière complète.			
Communication	Le dossier de conception ne comprend pas de faute d'orthographe. La mise en page, la structure des textes et leur contenu permettent une lecture facile et une compréhension totale du dossier.			

Pondération	/100

GRILLE D'EVALUATION

DEFENSE ORALE INDIVIDUELLE

Grille d'évaluation					
Critères	Indicateurs	Points	Acquisition		
			A	N-A	E-A
Technique et processus	L'élève prouve le développement d'une part équitable du projet. L'élève peut vulgariser avec aisance le fonctionnement général de son code. L'élève maitrise et peut restituer une explication claire d'un algorithme particulier. L'élève maitrise les détails techniques propre aux langages de programmation utilisés. L'élève est capable de proposer une correction de son code et/ou une ou plusieurs fonctionnalité(s) supplémentaire(s).				

Pondération	/100

FEUILLE DE DÉFINITION DU PROJET

Nom des élèves : ……

……

……

Choix du projet (cochez le projet choisi)**:**

Date :

Signature des élèves

Signature du professeur